



Vercel 部署与生产环境优化

从本地代码到全球化发布的完整指南

全流程实战指南

课前挑战：面试中的经典部署问题



Next.js 项目中，
"NEXT_PUBLIC_" 前缀的
环境变量有何特殊作用？



如何不重新构建代码，实
现生产环境的秒级回滚
(Instant Rollback)？

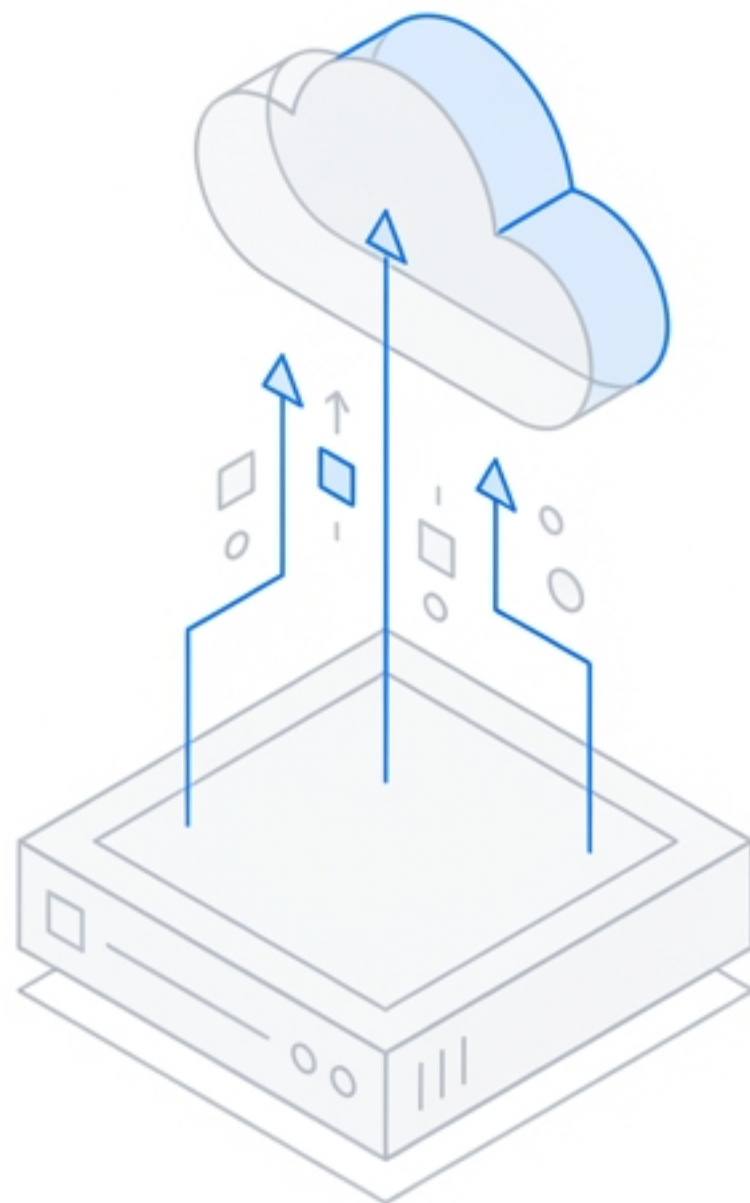


Next.js 的 Preview
Deployment 是如何改变
团队协作流程的？

本课程将为你揭晓这些问题的标准答案。

课程目录

- 1 前言：为什么选择 Vercel?
- 2 实战：零配置部署流程
- 3 核心：环境变量的安全机制
- 4 进阶：域名与 HTTPS 配置
- 5 协作：预览部署 workflow
- 6 运维：秒级回滚与监控



前端部署的终极方案

传统部署（繁琐）



买服务器、配 Nginx、申请证书、
写 CI/CD 脚本...

Vercel（自动化）



极致简单，自动运维

- ✓ Serverless: 自动扩容
- ✓ Edge Network: 全球 CDN
- ✓ Image Optimization: 自动压缩
- ✓ Cost: 个人项目免费

零配置部署实战

1. 准备 (Preparation)



- 确保 npm run build 本地通过
- 提交代码至 GitHub (Main branch)

2. 导入 (Import)

登录 Vercel
Dashboard

Import
Repository

Add New
Project



3. 智能识别 (Auto-Detect)

```
Framework Preset: Next.js  
Build Command: next build  
Output Directory: .next
```


环境变量配置

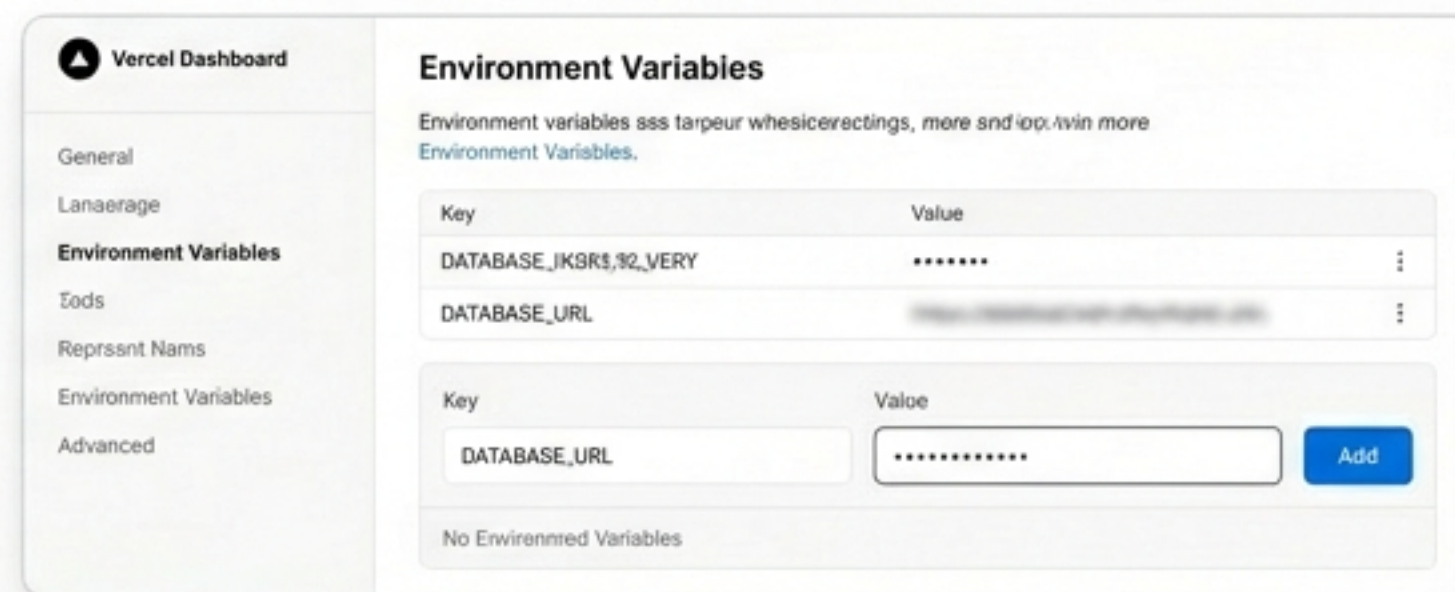
部署中唯一需要手动干预的步骤

The Reason



.env.local 被 .gitignore 忽略，Vercel 无法直接读取本地密钥。

The Solution



将本地内容复制到 Vercel Dashboard 的 Environment Variables 面板。

⚠ 注意：私密密钥 (Secrets) 和公开变量 (NEXT_PUBLIC_) 都必须在此配置。

深入解析：环境变量的作用域

Server Side (app/api/...)

```
const dbUrl = process.env.DATABASE_URL;  
console.log(dbUrl); // ✅ 输出连接串
```

可访问所有变量（安全）

Client Side (Components)

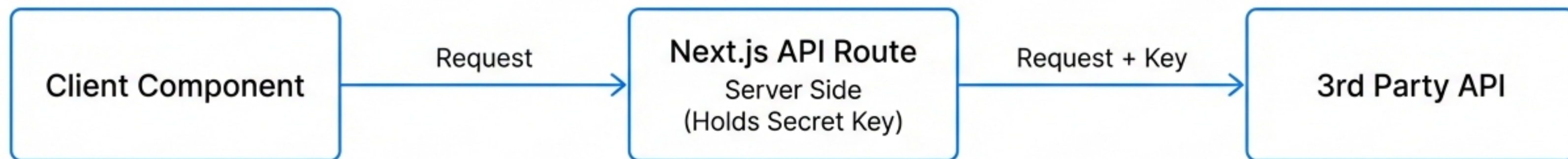
```
// ✅ 可访问公开信息  
const site = process.env.NEXT_PUBLIC_SITE_NAME;  
// ❌ undefined（安全保护）  
const secret = process.env.NEXTAUTH_SECRET;
```

仅可访问 NEXT_PUBLIC_ 前缀变量

最佳实践：混合架构下的密钥保护

Problem: 客户端需要数据，但第三方 API 需要私密 Key

Middleman Strategy



app/api/weather/route.ts

```
export async function POST(request: Request) {  
  const apiKey = process.env.WEATHER_API_KEY; // ✅ Server only  
  const res = await fetch(`https://api.weather.com?key=${apiKey}`);  
  return NextResponse.json(await res.json());  
}
```


生产环境配置：域名与 HTTPS

1. 添加域名 (Add Domain)

mars86.com

Dashboard -> Settings -> Domains

2. DNS 配置 (DNS Config)

A Record  76.76.21.21

CNAME  cname.vercel-dns.com

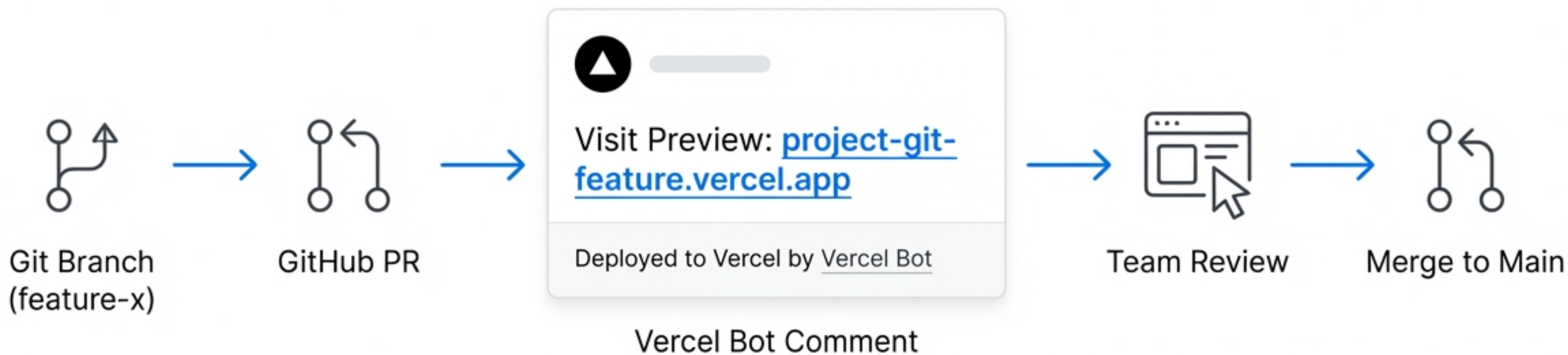
3. 自动 HTTPS (Automatic HTTPS)

DNS 生效后，自动部署 Let's Encrypt 证书。支持自动续期（Auto-renewal）。



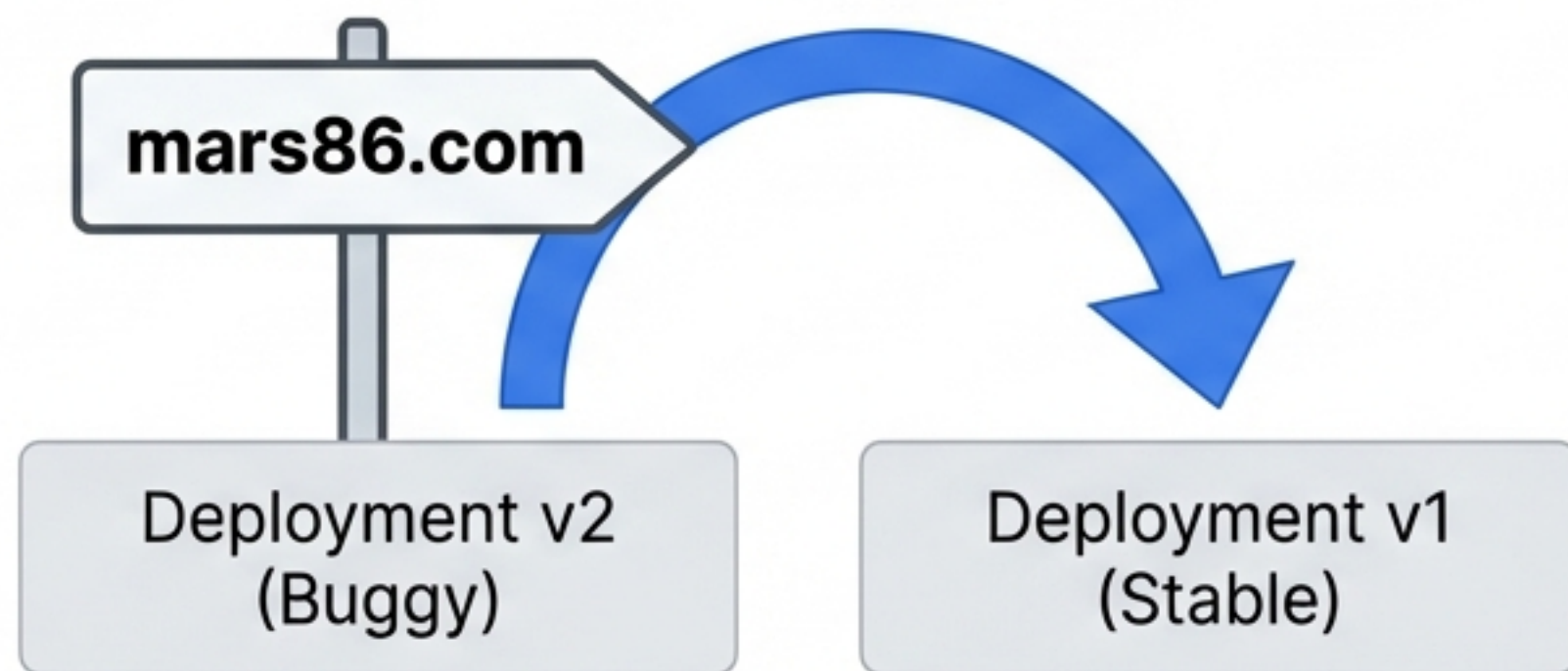
团队协作：预览部署 (Preview Deployments)

改变前端开发流程的杀手级功能



环境隔离 (Isolation)：这里的环境与线上完全隔离，无需搭建测试服。

生产环境救火：秒级回滚 (Instant Rollback)



场景：线上出现严重 Bug，急需恢复。

操作：Dashboard -> Deployments -> Select Version -> Instant Rollback

耗时：< 1 秒

原理：Immutable Deployments（不可变部署）。每次部署生成全新文件，回滚只需改变指向。

监控与成本优化

性能监控 (Speed Insights)



基于真实用户数据 (RUM)

关注指标: LCP (加载速度), INP (交互延迟)

```
npm install @vercel/speed-insights
```

日志与成本 (Logs & Cost)

```
[INFO] GET /api/data - 200 OK  
[ERROR] Failed to fetch data  
[INFO] Cache hit for /images/hero.png  
[INFO] GET /about - 304 Not Modified
```

Logs: 实时查看服务端 console.log

Tips: 优化 next/image 使用, 为 API Route 设置 Cache-Control

课程总结

- ✓ Code: 本地开发 (Local Development)
- ✓ Deploy: 零配置与变量 (Zero Config + Env Vars)
- ✓ Production: 域名与证书 (Domain + HTTPS)
- ✓ Maintain: 协作、回滚与监控 (Preview -> Rollback -> Monitor)

你现在拥有的, 不再是一个简单的 *Demo*, 而是一套可以支撑真实商业产品的现代化工程体系。

Thank You

Q & A